Projeto Integrador Avançado

Relatório Técnico

Pipeline de Dados, Modelagem Preditiva, Visão Computacional e Dashboard

Adriel Soares

Outubro de 2026

Sumário

1 Resumo executivo **1**

2 Escopo, fontes e rastreabilidade **1**

2.1 Objetivo geral 2

2.2 Objetivos específicos 2

3 Contexto industrial e dados **2**

3.1 Estrutura do conjunto de dados 2

3.2 Faixas de referência citadas no enunciado 3

4 Arquitetura implementada **4**

4.1 Organização funcional do repositório 4

5 Pipeline de dados e pré-processamento **5**

5.1 Inspeção estrutural 5

5.2 Tratamento de dados ausentes 6

5.3 Tratamento de outliers 6

5.4 Engenharia de atributos temporais 6

5.5 Análise exploratória (EDA) 7

6 Modelagem preditiva de telemetria **7**

6.1 Variável-alvo e classes 7

6.2 Modelo supervisionado 8

6.3 Pré-processamento dentro do modelo 8

6.4 Detecção não supervisionada 8

6.5 Avaliação e métricas 9

7 Simulador de telemetria **9**

7.1 Regimes e perfis gerados 10

8 Visão computacional e processamento de imagens **10**

8.1 Inspeção geométrica 10

8.2 Inspeção térmica 11

8.3 Pipeline de processamento e treinamento de imagens 11
8.4 Limitações e riscos da inspeção por imagem 12

9 API REST e validação de dados **12**
9.1 Endpoints identificados 12

10 Persistência e histórico **13**

11 Dashboard e visualização **13**

12 Validação documentada **14**

13 MLOps, monitoramento e governança **14**
13.1 Privacidade, ética e cibersegurança 15

14 Limitações conhecidas **15**

15 Conclusão **16**

1 Resumo executivo

Este relatório documenta a solução de monitoramento preditivo desenvolvida para o cenário acadêmico da AeroMotion Dynamics. A documentação foi elaborada a partir do enunciado do Projeto Integrador e dos arquivos e da descrição técnica disponíveis no repositório do projeto. O sistema combina telemetria industrial, engenharia de atributos temporais, classificação supervisionada, detecção de anomalias, processamento de imagens, API REST, persistência histórica e dashboard.

O conjunto original de telemetria descrito no enunciado contém 12.000 registros e representa quatro bancadas de teste, identificadas como TURB-101 a TURB-104. A aplicação implementada recebe leituras do simulador, constrói atributos temporais, executa os artefatos de Random Forest e Isolation Forest quando disponíveis, registra previsões em SQLite e apresenta resultados em uma interface Streamlit com gráficos Plotly. A visão computacional é organizada em módulos para inspeção geométrica e segmentação de regiões térmicas.

É essencial distinguir os requisitos acadêmicos dos resultados efetivamente documentados no repositório. O enunciado solicita experimentos comparativos, métricas de classificação, análise exploratória e procedimentos de pré-processamento; o README documenta uma validação controlada do simulador e da API, mas não fornece todos os resultados numéricos da EDA nem uma tabela completa de métricas de validação independente do classificador. Por isso, este relatório não inventa valores de correlação, métricas globais ou desempenho em equipamentos reais.

2 Escopo, fontes e rastreabilidade

O documento considera exclusivamente duas fontes:

1. o PDF do enunciado *Projeto Integrador Avançado: IA e Análise de Dados em Sistemas Industriais*;
2. o repositório `https://github.com/adrielsoares2309/Projeto_Aero`, em particular o README e os módulos da API, do simulador, do dashboard e da visão computacional.

O PDF define os objetivos acadêmicos, o dicionário de dados, as etapas e a rubrica de avaliação. O repositório é a fonte para descrever a implementação identificável no código. Quando uma atividade aparece no enunciado, mas não há evidência suficiente de execução no repositório consultado, ela é apresentada como requisito ou procedimento previsto, e não como resultado comprovado.

2.1 Objetivo geral

Desenvolver uma solução demonstrável de ponta a ponta para monitoramento de condições industriais, com processamento de telemetria, predição de estados de falha, identificação de anomalias, inspeção de imagens e visualização operacional.

2.2 Objetivos específicos

- inspecionar e tratar dados de sensores, preservando a relação temporal por equipamento;
- derivar atributos estatísticos em janelas de uma e seis horas e um indicador de severidade dinâmica;
- classificar quatro estados: Normal, Desgaste de Rolamento, Desbalanceamento do Rotor e Cavitação/Superaquecimento;
- utilizar Isolation Forest para sinalização não supervisionada de padrões atípicos;
- processar imagens para análise de contornos geométricos e regiões candidatas a hotspots térmicos;
- disponibilizar inferência e consulta histórica por API;
- exibir telemetria, previsões, probabilidades e estatísticas em dashboard;
- separar explicitamente resultados simulados de medições reais.

3 Contexto industrial e dados

O cenário proposto descreve quatro bancadas instrumentadas por sensores IoT, com coleta a cada dez minutos. As falhas-alvo incluem desgaste progressivo de rolamentos, desbalanceamento mecânico/aerodinâmico e cavitação combinada com superaquecimento. O enunciado também destaca ruído de sensores, perdas temporárias de pacotes e necessidade de inspeção óptica/térmica.

3.1 Estrutura do conjunto de dados

O arquivo de entrada indicado no enunciado é `dados_sensores_aeromotion.csv`, com 12.000 registros e 11 colunas.

Campo	Tipo / unidade	Descrição
`timestamp`	Data/hora	Instante da amostragem, previsto em intervalos de dez minutos.
`id_equipamento`	Categórico	Identificador TURB-101 a TURB-104.
`modo_operacao`	Categórico	Cruzeiro, Carga_Media ou Carga_Alta.
`horas_operacao_acumuladas`	Horas	Tempo acumulado de funcionamento.
`temperatura_mancal_c`	°C	Temperatura do mancal; o enunciado informa NaNs e valores espúrios como 999,9.
`vibracao_rms_mms`	mm/s RMS	Vibração global do eixo; o enunciado cita NaNs e valores espúrios como $-99,0$.
`pressao_oleo_bar`	bar	Pressão de lubrificação; podem existir ausentes.
`rotacao_rpm`	rpm	Velocidade angular do rotor.
`emissao_acustica_db`	dB	Indicador de ruído ultrassônico associado à fricção mecânica.
`corrente_eletrica_amp`	A	Corrente consumida pelo acionamento.
`tipo_falha`	Inteiro / alvo	0 Normal; 1 Desgaste de Rolamento; 2 Desbalanceamento do Rotor; 3 Cavitação/Superaquecimento.

3.2 Faixas de referência citadas no enunciado

As faixas a seguir são referências do documento acadêmico, não uma especificação de segurança para operação real.

Variável	Faixa normal descrita
Temperatura do mancal	45–85 °C
Vibração RMS	1,0–4,5 mm/s
Pressão do óleo	3,5–5,2 bar
Rotação	2300–3600 rpm
Emissão acústica	40–68 dB
Corrente elétrica	18–42 A

4 Arquitetura implementada

A arquitetura documentada no repositório pode ser resumida pelo fluxo abaixo.

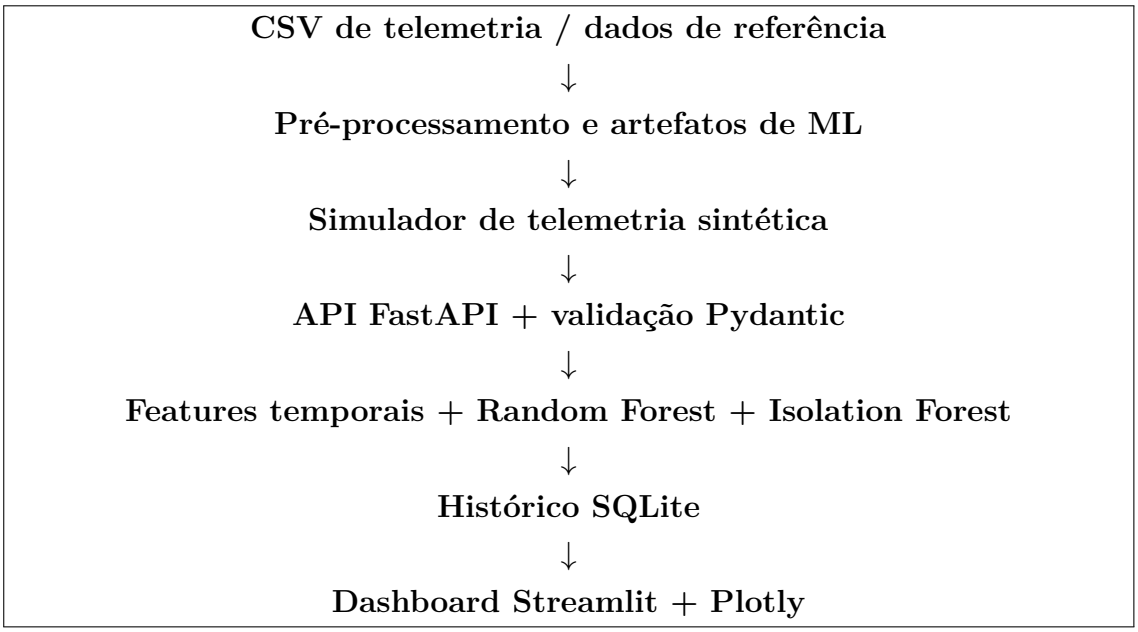


Figura 1: Fluxo funcional do sistema de monitoramento preditivo.

A inspeção de imagens segue um fluxo paralelo: imagem de entrada, identificação do tipo geométrico ou térmico, processamento OpenCV, extração de regiões/contornos e disponibilização dos resultados ao módulo de interface. A visão computacional não deve ser confundida com o classificador de falhas baseado em telemetria: são caminhos analíticos diferentes.

4.1 Organização funcional do repositório

Caminho	Responsabilidade identificada
`api/main.py`	Aplicação FastAPI e endpoints de saúde, predição, lote, histórico, estatísticas e controle de regime.
`api/schemas.py`	Contratos de entrada Pydantic, validação de campos, equipamentos, regimes e metadados.
`api/predictor.py`	Carregamento dos artefatos, construção das features temporais e inferência.
`api/history.py`	Persistência das leituras e previsões em SQLite, com migração aditiva de colunas.
`simulator.py`	Geração de telemetria sintética, dinâmica física simplificada, troca de regime e envio à API.
`frontend/app.py`	Interface Streamlit, controles de regime e integração com módulos de gráficos, componentes e visão.
`vision/geometry.py`	Suavização, Canny, contornos e cálculo de características geométricas.
`vision/thermal.py`	Segmentação HSV de regiões candidatas a hotspots.
`vision/pipeline.py`	Coordenação do processamento e do treinamento/avaliação de imagens.
`requirements.txt`	Dependências Python para análise de dados, ML, API, dashboard e OpenCV.

5 Pipeline de dados e pré-processamento

O enunciado requer inspeção estrutural, análise de timestamps, quantificação de ausências por equipamento, tratamento de ausentes e outliers e criação de atributos temporais. As decisões abaixo descrevem o procedimento metodológico previsto no enunciado e a interface de atributos confirmada no código da API. Não se afirma que todos os gráficos exploratórios ou todos os métodos propostos tenham sido executados, pois os resultados numéricos completos não constam do README.

5.1 Inspeção estrutural

A primeira etapa consiste em verificar dimensões, nomes de colunas, tipos de dados, duplicatas, valores ausentes e coerência dos timestamps. A série deve ser examinada separadamente por `id_equipamento`, pois misturar equipamentos na interpolação ou no

cálculo de janelas poderia contaminar o histórico de uma bancada com leituras de outra.

5.2 Tratamento de dados ausentes

O PDF recomenda interpolação temporal linear ou spline por equipamento para lacunas pontuais. A escolha deve respeitar a continuidade do processo e não criar valores artificiais em lacunas longas sem justificativa. A quantidade e a proporção de ausentes devem ser registradas antes e depois do tratamento, por coluna e por equipamento.

O contrato do modelo contém um pipeline de pré-processamento com imputação, `RobustScaler` e codificação one-hot, conforme o README. A API envia os atributos como DataFrame para esse pipeline e evita repetir o pré-processamento. O material consultado não especifica todos os parâmetros da imputação nem apresenta a tabela de ausências antes/depois; esses detalhes devem ser extraídos do script/notebook de treinamento para compor uma auditoria reprodutível.

5.3 Tratamento de outliers

O enunciado cita valores fisicamente implausíveis, como temperatura de 999,9 °C e vibração de $-99,0$ mm/s. Recomenda-se identificá-los por limites físicos e/ou Z-score móvel, distinguir erro de medição de evento real e registrar a regra aplicada. A substituição não deve apagar silenciosamente sinais legítimos de falha.

A simulação documentada no README utiliza limites de geração derivados dos extremos observados nos dados tratados: temperatura de 40,0 a 119,6 °C; vibração de 0,08 a 15,17 mm/s; pressão de 0,68 a 5,59 bar; rotação de 2278,2 a 3641,4 rpm; acústica de 34,15 a 104,0 dB; corrente de 15,35 a 62,54 A. O repositório alerta que esses intervalos são limites do gerador sintético e não limites de segurança industrial.

5.4 Engenharia de atributos temporais

O contrato de features em `api/predictor.py` contém 17 atributos de entrada:

- modo de operação e horas acumuladas;
- temperatura, vibração, pressão, rotação, emissão acústica e corrente;
- média e desvio-padrão de temperatura em janelas de 1 h e 6 h;
- média e desvio-padrão de vibração em janelas de 1 h e 6 h;
- severidade dinâmica.

A severidade dinâmica é definida no enunciado como a razão entre vibração RMS e rotação:

$$S_d = \frac{\text{vibracao_rms_mms}}{\text{rotacao_rpm}}.$$

A implementação mantém buffers por equipamento e restaura histórico recente para compor as janelas. O README informa que, até haver duas leituras na janela de uma hora, o dashboard sinaliza a condição de aquecimento e o imputador do pipeline trata os atributos ainda indisponíveis. Esse comportamento é relevante no início do monitoramento e após reinício sem histórico suficiente.

5.5 Análise exploratória (EDA)

O enunciado solicita estatística descritiva, taxa de ausência por equipamento, séries temporais, matriz de correlação cruzada, pairplots e comparação dos regimes Cruzeiro, Carga_Media e Carga_Alta. A EDA deve investigar:

1. distribuição, escala e assimetria das variáveis;
2. valores ausentes e sua concentração temporal/equipamento;
3. outliers e possíveis artefatos instrumentais;
4. correlações entre temperatura, vibração, pressão, rotação, acústica e corrente;
5. mudanças de distribuição entre regimes operacionais;
6. diferenças entre as classes de `tipo_falha`.

Limite documental: o repositório consultado não fornece uma tabela final com estatísticas descritivas, matriz de correlação ou coeficientes numéricos de EDA. Assim, não é tecnicamente correto afirmar que uma variável apresentou determinada correlação ou que uma classe possui uma distribuição específica sem consultar os resultados efetivamente gerados pelo notebook.

6 Modelagem preditiva de telemetria

6.1 Variável-alvo e classes

A classificação possui quatro estados definidos no enunciado e usados pela API:

Código	Classe
0	Normal
1	Desgaste de Rolamento
2	Desbalanceamento do Rotor
3	Cavitação/Superaquecimento

6.2 Modelo supervisionado

O README identifica `random_forest.joblib` como classificador selecionado. A classe `ModelService`, em `api/predictor.py`, carrega esse artefato e verifica se os nomes das features esperadas coincidem com o contrato auditado de 17 atributos. Se o arquivo do classificador não existir ou o contrato for incompatível, o serviço sinaliza a indisponibilidade do modelo.

O PDF sugere comparar dois classificadores multiclasse, como Random Forest versus XGBoost ou LightGBM, ajustar hiperparâmetros com GridSearchCV ou RandomizedSearchCV e avaliar o Recall por classe. Embora o arquivo de dependências inclua Scikit-learn, XGBoost, LightGBM e imbalanced-learn, a presença de uma biblioteca em `requirements.txt` não comprova, por si só, que todos os modelos foram treinados e comparados. A evidência consultada identifica o Random Forest como modelo implantado, mas não apresenta a tabela integral de comparação de métricas.

6.3 Pré-processamento dentro do modelo

Segundo o README, o pipeline serializado inclui imputação, `RobustScaler` e codificação one-hot. A API não aplica uma segunda transformação manual, evitando diferenças entre o processamento do treinamento e o da inferência. Essa consistência é importante porque o modelo recebe atributos contínuos e uma variável categórica de regime.

6.4 Detecção não supervisionada

O sistema carrega opcionalmente `isolation_forest.joblib` e `preprocessador_anomalias.joblib`. Quando ambos estão disponíveis, a inferência inclui a saída do Isolation Forest. Essa sinalização é distinta da classe supervisionada: uma amostra pode ser considerada anômala pelo detector sem que isso determine automaticamente qual classe de falha o classificador deve prever.

6.5 Avaliação e métricas

O enunciado exige matriz de confusão 4×4, acurácia, precisão, Recall e F1-score macro e weighted, com prioridade ao Recall por classe de falha. O Recall é especialmente relevante porque falsos negativos podem deixar uma condição de falha sem alerta. Para completar a avaliação formal, devem ser incluídos os resultados reais do conjunto de teste e a metodologia de separação temporal treino/teste de 70%/30%.

Resultados numéricos disponíveis no README: a validação controlada documentada é uma avaliação da reação do modelo à telemetria sintética por regime, não uma estimativa independente de generalização em dados reais. Foram enviados 1.200 registros (100 ciclos × quatro equipamentos em cada um dos três regimes) à API real, com seed 20261026. As previsões observadas foram:

Regime	Geradas N/A	Classes previstas (0/1/2/3)	Inferência média
Cruzeiro	380/20	400/0/0/0	81,55 ms
Carga_Media	280/120	366/16/18/0	80,23 ms
Carga_Alta	160/240	149/167/77/7	80,27 ms

N/A significa perfil sintético nominal/anormal gerado, e não classe prevista. Os resultados mostram que o classificador respondeu de maneira diferente conforme os sinais produzidos, mas não devem ser interpretados como acurácia ou Recall de teste. Em Cruzeiro, todas as 400 previsões foram Normal; em Carga_Media foram previstas 366 amostras como Normal, 16 como Desgaste de Rolamento e 18 como Desbalanceamento; em Carga_Alta foram previstas 149 como Normal, 167 como Desgaste, 77 como Desbalanceamento e 7 como Cavitação/Superaquecimento.

O README relata ainda que, em Carga_Alta, o perfil sintético B foi classificado como Desgaste de Rolamento em 86 de 86 leituras; o perfil C foi previsto como Desbalanceamento em 55 de 75 e como classe 1 em 20 de 75; o perfil A foi majoritariamente previsto como Normal (41 de 79). São resultados de um teste controlado, com uma semente, e não evidência de desempenho em turbinas reais.

7 Simulador de telemetria

O simulador mantém estado por equipamento e gera dados sintéticos para os quatro equipamentos. A documentação do repositório descreve inércia temporal, ruído de medição correlacionado AR(1), oscilações operacionais e drift lento. As constantes de tempo infor-

madas são 20 s para carga, 12 s para rotação, 18 s para vibração, 90 s para temperatura, 35 s para pressão, 20 s para acústica e 12 s para corrente.

7.1 Regimes e perfis gerados

Regime	Alvo de carga	Nominal	Anormal	Intensidade do desvio
Cruzeiro	0,25	95%	5%	30%
Carga_Media	0,55	70%	30%	60%
Carga_Alta	0,90	40%	60%	100%

Os alvos de equilíbrio foram derivados, segundo o README, das médias das leituras normais por regime no dataset tratado. As médias de referência são:

Regime	Temp.	Vib.	Pressão	RPM	Acústica	Corrente
Cruzeiro	51,9	1,81	3,81	2400,7	44,0	20,05
Carga_Media	61,9	2,50	4,20	3000,5	52,0	27,98
Carga_Alta	72,0	3,80	4,80	3500,6	62,0	38,02

Unidades: temperatura em °C, vibração em mm/s, pressão em bar, rotação em rpm, emissão acústica em dB e corrente em A.

Os perfis anormais são cenários sintéticos: A representa sobrecarga térmica, B representa solicitação mecânica e vibração e C representa desvio combinado. O README explicita que uma amostra anormal é uma intenção do gerador, não um rótulo confirmado de falha. O regime selecionado também não define a classe prevista pelo modelo.

8 Visão computacional e processamento de imagens

A visão computacional é implementada em módulos próprios, utilizando OpenCV e NumPy. A arquitetura do repositório diferencia inspeção geométrica e inspeção térmica.

8.1 Inspeção geométrica

O módulo `vision/geometry.py` define parâmetros para suavização Gaussiana, limiares Canny e área mínima de contorno. O fluxo inclui:

1. carregar e validar a imagem;

2. converter para escala de cinza;
3. aplicar suavização Gaussiana para reduzir ruído;
4. aplicar Canny para obter bordas;
5. extrair contornos externos;
6. calcular área, perímetro, circularidade, caixa delimitadora, centroide e razão de aspecto.

A circularidade é calculada por:

$$C = \frac{4\pi A}{P^2},$$

em que A é a área do contorno e P é o perímetro. Para uma circunferência ideal, o indicador aproxima-se de 1. O código alerta que discretização e segmentação afetam a medida; portanto, circularidade isolada não basta para declarar uma peça defeituosa. A validação contra referências geométricas depende de imagens de referência adequadas e de limites definidos para o componente inspecionado.

8.2 Inspeção térmica

O módulo `vision/thermal.py` converte imagens BGR para HSV e aplica limiares configuráveis para segmentar regiões candidatas a hotspots. A configuração inicial procura regiões vermelhas em dois intervalos de matiz, porque o vermelho atravessa o limite circular do canal H no HSV do OpenCV. A máscara pode passar por operações morfológicas de abertura e fechamento; regiões abaixo da área mínima são descartadas.

O método identifica regiões de cor na imagem, não calcula diretamente temperatura em graus Celsius. Para converter cores em temperatura absoluta seriam necessárias informações de calibração, emissividade, escala da câmera e metadados térmicos, não demonstrados pelo processamento descrito. Portanto, a saída deve ser interpretada como segmentação visual candidata a região quente, não como medição térmica calibrada.

8.3 Pipeline de processamento e treinamento de imagens

O módulo `vision/pipeline.py` identifica o tipo de imagem por parâmetro explícito ou por termos nos nomes dos diretórios/arquivos. Quando o tipo é ambíguo, a imagem é devolvida como não classificada e o código solicita uma indicação explícita. O pipeline processa imagens geométricas ou térmicas e grava artefatos de saída em diretórios distintos.

O módulo também importa RandomForestClassifier e funções de divisão treino/teste e métricas do Scikit-learn para o treinamento/avaliação da classificação de imagens. Contudo, não é possível declarar uma acurácia, precisão, Recall ou quantidade de imagens de treinamento sem os resultados efetivos da execução. Para documentar o treinamento de forma reproduzível, a versão final do relatório deve registrar: número de imagens por classe, fonte e licença das imagens, regra de rotulagem, divisão treino/validação/teste, sementes aleatórias, atributos usados, hiperparâmetros, matriz de confusão e métricas obtidas.

8.4 Limitações e riscos da inspeção por imagem

A performance depende de iluminação, foco, perspectiva, resolução, paleta térmica e domínio de captura. Limiares HSV podem variar entre câmeras ou paletas. A análise de contornos requer referências dimensionais ou geométricas para decidir tolerâncias. Sem conjunto de imagens rotuladas e avaliação independente, o pipeline deve ser entendido como mecanismo de processamento e extração de evidências visuais, e não como certificação autônoma de qualidade industrial.

9 API REST e validação de dados

A API é implementada com FastAPI e Pydantic. Os schemas restringem equipamentos a TURB-101–TURB-104, regimes a Cruzeiro, Carga_Media e Carga_Alta e definem limites numéricos de entrada para as leituras. Os timestamps são normalizados para remover informação de fuso horário, de acordo com o contrato implementado.

9.1 Endpoints identificados

Endpoint	Finalidade
`GET /health`	Informa se o serviço está online/degradado e se os artefatos foram carregados.
`POST /predict`	Executa inferência para uma leitura.
`POST /predict-batch`	Processa uma lista de leituras, limitada no schema a 1–500 itens.
`GET /predictions`	Consulta histórico, com limite e filtro opcional por equipamento.
`GET /stats`	Retorna estatísticas do histórico de predições.

Endpoint	Finalidade
`GET /regime`	Retorna regime selecionado/ativo e estado do simulador.
`POST /regime`	Solicita alteração do regime operacional do simulador.
`GET /regimes`	Lista regimes e rótulos disponíveis.

Embora o enunciado peça uma API assíncrona, os handlers mostrados em `api/main.py` são funções `def` síncronas. Assim, a implementação é uma API FastAPI, mas não se deve afirmar que os endpoints de inferência são assíncronos apenas por utilizarem FastAPI. A documentação OpenAPI/Swagger pode ser acessada, em execução local, em http://127.0.0.1:8000/docs.

10 Persistência e histórico

O módulo `api/history.py` utiliza SQLite para persistir leituras, resultados, classe prevista, indicação de anomalia, duração de inferência, regime operacional, origem da leitura, probabilidades e metadados dos perfis sintéticos. O código preserva o esquema existente e adiciona colunas por migração aditiva, em vez de apagar o histórico anterior. Há índice para equipamento e timestamp, além de índice para regime operacional.

A separação entre `origem_leitura`, `perfil_sintetico`, `perfil_anomalia` e `tipo_falha_pre` evita tratar a intenção do simulador como se fosse a decisão do classificador. Essa distinção é fundamental para interpretar corretamente gráficos e estatísticas de avaliação.

11 Dashboard e visualização

O frontend é construído com Streamlit e Plotly. O código importa módulos específicos de gráficos, componentes, ícones SVG e integração com a API. O design descrito no código utiliza fundo predominantemente branco, texto azul-escuro, gráficos com fundo branco e ícones SVG, sem emojis.

O dashboard permite controlar os três regimes de operação e acompanha os dados retornados pela API. O README descreve cartões de telemetria, histórico, probabilidades, estatísticas e uma seção de avaliação da reação do modelo por regime. Os gráficos de desempenho devem sempre diferenciar:

- proporção nominal/anormal produzida pelo simulador;
- classe efetivamente prevista pelo modelo;

- probabilidades retornadas pelo classificador;
- saída independente do detector Isolation Forest;
- origem simulada ou real da leitura.

O regime operacional não deve ser usado como substituto da previsão. Uma carga elevada não é, por si só, uma falha; da mesma forma, um perfil sintético anormal não garante que o modelo identifique uma classe de falha.

12 Validação documentada

O README descreve uma validação controlada executada em 26 de outubro de 2026, utilizando a classe real `TelemetrySimulator`, seed 20261026, 100 ciclos por regime e quatro equipamentos, com envio por `/predict-batch`. Foram geradas 1.200 leituras, com IDs de histórico de 25948 a 27147. Segundo o README, as leituras permaneceram finitas e dentro das faixas de geração, e as proporções planejadas por regime foram reproduzidas.

A mesma execução confirmou a troca de regime sem reiniciar o processo do simulador e sem apagar o histórico. Uma segunda tentativa de iniciar o simulador foi recusada pelo mecanismo de lock. O histórico preexistente de 25.828 linhas sem perfil foi preservado e o total passou a 27.147 linhas.

O README ressalva que os timestamps foram gerados em lote e o relógio de parede não foi aguardado durante 25 minutos. Portanto, essa execução valida integração e comportamento controlado do sistema, mas não equivale a um ensaio de duração real nem a uma validação em equipamentos físicos. Nenhum comando foi enviado a CLPs reais.

13 MLOps, monitoramento e governança

O enunciado solicita exportação dos modelos e escaladores, monitoramento de data drift e concept drift em janela operacional de 12 meses e atenção à explicabilidade. O repositório já utiliza artefatos serializados via Joblib e verifica a compatibilidade das features esperadas pelo classificador. O plano operacional deve evoluir com:

1. registro de versão do modelo, data de treinamento, versão dos dados e configuração;
2. monitoramento da distribuição de sensores, taxas de ausentes e frequência de classes previstas;

3. detecção de mudanças estatísticas nas entradas (data drift) e de perda de qualidade confirmada por rótulos posteriores (concept drift);
4. reavaliação periódica com dados rotulados e aprovação antes de substituir o modelo;
5. alertas para falha de carregamento de artefatos, latência e indisponibilidade da API;
6. trilha de auditoria de previsões e origem das leituras.

Esses itens constituem recomendações alinhadas aos requisitos do enunciado; o README não demonstra que exista um sistema completo de monitoramento de drift em produção.

13.1 Privacidade, ética e cibersegurança

O PDF solicita parecer sobre impactos trabalhistas, requalificação profissional, LGPD e mascaramento de rostos, classificação de risco no EU AI Act e segurança da rede OT/IT. Para uso real, o tratamento de imagens deve limitar a coleta ao necessário, evitar identificar operadores quando isso não for indispensável e definir controles de acesso, retenção e finalidade. A integração com redes industriais exige autenticação, segmentação de rede, gestão de segredos, atualização de dependências, logs e validação dos dados recebidos.

A aplicação descrita no repositório é demonstrativa. O README afirma explicitamente que as leituras são sintéticas e que nenhum comando é enviado a PLCs. Não há evidência, nas fontes consultadas, de certificação regulatória, implantação em produção ou validação de segurança funcional.

14 Limitações conhecidas

- Os resultados de simulação não são medições reais de campo.
- As probabilidades do classificador são saídas do modelo, não confirmação de defeito físico.
- Os perfis nominais/anormais do simulador não são rótulos de falha confirmada.
- Os valores-limite de geração sintética não são limites de segurança industrial.
- A dinâmica temporal do simulador é uma hipótese de modelagem explicitada no README; o conjunto original é descrito como essencialmente independente dentro de cada regime.

- O README não apresenta uma tabela completa de métricas independentes de teste, nem os valores numéricos da EDA.
- A inspeção térmica por HSV identifica regiões coloridas candidatas a hotspots; não converte cor diretamente em temperatura calibrada.
- O desempenho de classificação de imagens não pode ser afirmado sem métricas e conjunto de teste documentados.
- A API possui handlers síncronos no código consultado, apesar do requisito acadêmico mencionar API assíncrona.

15 Conclusão

O projeto reúne os principais componentes de uma solução industrial de IA: telemetria sintética com dinâmica temporal, API de inferência, classificador Random Forest, detector opcional Isolation Forest, armazenamento histórico SQLite, dashboard Streamlit/Plotly e módulos OpenCV para análise geométrica e térmica. O contrato de inferência explicita 17 features e mantém o pré-processamento dentro do pipeline serializado, reduzindo o risco de divergência entre treinamento e inferência.

A validação registrada no README demonstra integração controlada entre simulador, API, persistência e dashboard, com tempos médios de inferência próximos de 80 ms no ensaio descrito. Esses números não são métricas de acurácia nem demonstram desempenho em equipamentos reais. Para fechar a documentação acadêmica com rigor, devem ser anexados os resultados executados do notebook de EDA, a comparação formal de classificadores no conjunto de teste, as métricas por classe e a avaliação quantitativa do treinamento de imagens. Onde tais resultados não estão nas fontes consultadas, este relatório preserva explicitamente a lacuna em vez de fabricar conclusões.

Referências documentais

1. AEROMOTION DYNAMICS S.A. *Projeto Integrador Avançado: IA e Análise de Dados em Sistemas Industriais.* Enunciado acadêmico fornecido em PDF, 9 páginas.
2. SOARES, Adriel. *Projeto AeroMotion Dynamics.* Repositório de código-fonte. Disponível em: https://github.com/adrielsoares2309/Projeto_Aero. Acesso em: 27 out. 2026.